TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

BASE DE DATOS II

TRABAJO PRÁCTICO — UNIDAD 5

Administración y seguridad de Food Store: roles, auditoría, incidentes y protección de datos personales

Integra las Clases 1 y 2 de la Unidad 5 — desarrollo con OpenCode, Kiro y Git

Asignatura	Base de Datos II — Unidad 5: Administración y Seguridad
Ubicación	Semana 9 de la cursada — integra el contenido de la Clase 1 (usuarios, roles, permisos, auditoría) y de la Clase 2 (amenazas y protección de datos personales)
Prerrequisitos	Haber cursado ambas clases de la Unidad 5. Contar con la base de Food Store poblada (Unidades 1 a 4) accesible en un entorno de desarrollo propio
Motor	PostgreSQL 16 o superior
Modalidad	Individual o en equipos de hasta tres integrantes, según indique la cátedra en el aula virtual
Herramientas obligatorias	OpenCode (asistente de IA de terminal), Kiro (especificación y planificación asistida) y Git (control de versiones)
Entrega	Repositorio Git con la estructura descrita en este documento, según el plazo publicado en el aula virtual
Ciclo lectivo	2026

Presentación del trabajo

Este Trabajo Práctico cierra la Unidad 5 y, con ella, integra en un mismo entregable todo lo trabajado a lo largo de las dos clases de Administración y Seguridad. La Clase 1 construyó un modelo de identidades: qué roles necesita Food Store, qué privilegios recibe cada uno bajo el principio de mínimo privilegio, y cómo empezar a auditar lo que ocurre sobre la base. La Clase 2 puso ese modelo a prueba: qué amenazas lo pueden vulnerar, y qué se debe hacer con los datos personales que Food Store almacena, incluyendo la frontera concreta para trabajar con inteligencia artificial sin exponerlos.

El trabajo se organiza en cinco partes que siguen ese mismo recorrido: primero se diseña e implementa el esquema de roles y permisos; luego se activa y prueba la auditoría; a continuación, se ejecuta un laboratorio central —un simulacro de incidente de acceso indebido, reconstruido con ayuda de IA y validado de forma crítica por el equipo—; después se aplica anonimización a un caso concreto de Food Store; y finalmente se consolida todo el proceso en una bitácora de uso de IA. No es un trabajo teórico: cada parte deja una evidencia

técnica verificable —un script que corre, un log que se puede leer, una consulta cuyo resultado se puede comparar.

Por qué se trabaja sobre la propia base de desarrollo
Todo lo que este trabajo pide —incluido el simulacro de incidente— se ejecuta exclusivamente contra la instancia de Food Store que cada estudiante o equipo tiene en su propio entorno de desarrollo. Ninguna consigna habilita, ni debe interpretarse como una habilitación, para probar técnicas de ataque contra sistemas de terceros, bases de otros equipos o cualquier servicio que no sea el propio. La ética profesional de la seguridad informática empieza por esta distinción.

Objetivos de aprendizaje

- Diseñar e implementar, con sentencias SQL reales, un esquema de roles y permisos que aplique el principio de mínimo privilegio a un caso concreto.
- Activar y poner a prueba mecanismos de auditoría, distinguiendo un registro técnico de un registro con vocación de rendición de cuentas.
- Reconstruir un incidente de seguridad a partir de evidencia de log, usando IA como acelerador de la hipótesis y validación humana como paso obligatorio antes de decidir.
- Aplicar técnicas de anonimización sobre datos de Food Store y verificar que los resultados agregados se mantienen equivalentes.
- Documentar de forma trazable —en Git y en una bitácora de IA— cada decisión técnica y cada interacción relevante con un asistente de inteligencia artificial.

Modalidad de trabajo: OpenCode, Kiro y Git

Como en los trabajos prácticos anteriores de la materia, este TP se resuelve con el flujo de trabajo asistido por IA que la cátedra viene utilizando: OpenCode como asistente de código de terminal, Kiro para la planificación por especificaciones, y Git como sistema de control de versiones. El uso de estas tres herramientas no es una formalidad: cada una deja evidencia concreta que forma parte de lo que se evalúa.

OpenCode

OpenCode se utiliza como el asistente de IA principal para escribir y revisar SQL, generar el script del simulacro de incidente, y participar en la reconstrucción del log de la Parte C. Toda interacción relevante con OpenCode —el prompt enviado y la respuesta obtenida— debe quedar registrada en la bitácora de IA (ver más abajo), junto con la validación o corrección humana que el equipo aplicó sobre esa respuesta.

Kiro

Antes de escribir el script de roles y permisos de la Parte A, el equipo redacta en Kiro una especificación breve: qué roles se necesitan, qué privilegios recibe cada uno y por qué. Esta especificación —el archivo de requirements y de design que Kiro genera— se entrega junto con el resto del trabajo. Planificar antes de escribir código es, en sí mismo, una aplicación del principio de mínimo privilegio: obliga a justificar cada permiso antes de otorgarlo, en lugar de otorgarlo primero y justificarlo después.

Git

El repositorio debe reflejar el avance del trabajo por partes, con historial de commits legible: no se acepta un único commit final con todo el contenido. Se sugiere un commit (o una serie corta de commits) por cada Parte del trabajo, con mensajes que describan qué se hizo y por qué, siguiendo las convenciones ya usadas en trabajos anteriores de la materia.

Bitácora de uso de IA — formato obligatorio
Se entrega un archivo BITACORA.md en la raíz del repositorio con una tabla de tres columnas: Prompt enviado a la IA, Respuesta relevante recibida (resumida, no es necesario copiar la respuesta completa) y Validación o corrección humana aplicada. Debe incluir, como mínimo, un registro por cada parte del trabajo en la que se haya usado OpenCode, y señalar explícitamente al menos un caso en el que la IA propuso algo incorrecto, inseguro o de más —y cómo el equipo lo detectó y corrigió.

El caso de Food Store: punto de partida

Este trabajo asume que el equipo ya cuenta, de la Clase 1, con una primera versión de roles.sql (los roles rol_app_lectura, rol_app_escritura, rol_soporte, rol_reportes, y los roles de login app_web y admin_datos) y con las funciones fn_autenticar y fn_resetear_password trabajadas en la Clase 2. Si esos artefactos no se conservaron o difieren de lo visto en clase, el equipo debe reconstruirlos como paso previo a la Parte A, dejando constancia de esa reconstrucción en la bitácora.

PARTE A. Diseño e implementación de roles y permisos de mínimo privilegio

El objetivo de esta parte es dejar, para Food Store, un esquema de permisos completo, verificado y justificado — no una copia de lo visto en clase, sino una versión revisada y, de ser necesario, ampliada por el propio equipo.

1. Redactar en Kiro una especificación breve (requirements + design) que releve los roles reales que necesita Food Store: como mínimo, los cuatro roles de grupo y los dos roles de login vistos en clase, más cualquier rol adicional que el equipo considere necesario para su versión del proyecto. Cada rol debe quedar justificado en una oración: qué operación habilita y por qué esa operación, y no otra.
2. Escribir (o revisar y ampliar) el archivo roles.sql con las sentencias CREATE ROLE, GRANT y REVOKE correspondientes. Incluir al menos un caso de permiso a nivel de columna (por ejemplo, sobre

usuario) y la sentencia ALTER DEFAULT PRIVILEGES que asegure que las tablas futuras hereden la misma política.

3. Verificar, con \du y con una consulta sobre information_schema.role_table_grants, que cada rol tiene exactamente los privilegios previstos en la especificación de Kiro —ni más, ni menos—. Adjuntar la salida de esa verificación.

4. Pedirle a OpenCode que audite roles.sql y proponga mejoras o correcciones. Registrar en la bitácora al menos un caso en el que la IA haya propuesto un permiso más amplio de lo necesario (por ejemplo, un GRANT ALL en lugar de un privilegio puntual), y explicar por qué el equipo decidió no aplicarlo tal cual.

Entregable de esta parte: *roles.sql actualizado, la especificación de Kiro (requirements y design), la salida de la verificación de permisos, y el registro correspondiente en BITACORA.md.*

PARTE B. Activación y prueba de la auditoría

Un esquema de permisos correcto no alcanza si nadie puede reconstruir después qué ocurrió sobre la base. Esta parte pone en funcionamiento, y prueba, un primer nivel de auditoría.

1. Activar, en la configuración del servidor de desarrollo, los parámetros de auditoría vistos en la Clase 1 (como mínimo log_connections, log_disconnections y log_statement en un nivel que registre operaciones de modificación). Documentar en un informe corto qué parámetros se modificaron y con qué valores.

2. Ejecutar, contra la base de Food Store, una secuencia breve de operaciones de prueba: al menos una lectura, una inserción, una actualización, y un intento de inicio de sesión fallido. Capturar el fragmento de log resultante.

3. Señalar, sobre ese fragmento, qué información es útil a los fines de auditoría (quién, qué, cuándo, desde dónde) y qué quedaría fuera del alcance de este mecanismo nativo si se necesitara mayor granularidad —por ejemplo, el detalle fila por fila que ofrecería la extensión pgaudit, a nivel de nociones.

4. Responder por escrito, en dos o tres oraciones: ¿el propio archivo de log es, en sí mismo, un activo sensible? ¿Quién debería poder leerlo, y con qué permisos del sistema operativo o del motor?

Entregable de esta parte: *El informe de configuración, el fragmento de log capturado, y las respuestas escritas de los puntos 3 y 4.*

PARTE C. Laboratorio: simulacro de incidente de acceso indebido

Esta es la parte central del trabajo: se reproduce, de forma controlada y exclusivamente contra el propio entorno de desarrollo, un incidente de seguridad completo, y se lo reconstruye con ayuda de IA aplicando lectura crítica sobre lo que la IA responde.

1. Diseñar y ejecutar un script (SQL, o SQL orquestado desde un script de shell o de Python, a elección del equipo) que reproduzca la siguiente secuencia sobre la propia base de Food Store: varios intentos fallidos de inicio de sesión contra fn_autenticar, seguidos de un inicio de sesión exitoso, seguidos de una lectura masiva de la tabla usuario, seguidos de un intento de otorgarle a un rol de bajo privilegio membresía en admin_datos (o de ejecutar una operación reservada a ese rol).
2. Verificar y dejar constancia de si ese último intento —el de escalamiento— efectivamente falla. Si no falla, señalarlo como un hallazgo: es evidencia de que el esquema de permisos de la Parte A tiene un defecto que debe corregirse antes de continuar.
3. Extraer el fragmento de log correspondiente a toda la secuencia y anonimizarlo antes de continuar: reemplazar mails, nombres y cualquier otro dato identificable por valores sintéticos, siguiendo el mismo criterio aplicado a usuario_anon en la Clase 2 —aun tratándose de un entorno de prueba, se practica el hábito de no exponer datos reales a un asistente de IA.
4. Entregar el log anonimizado a OpenCode, sin agregar contexto adicional al que el log ofrece, y pedirle que reconstruya qué ocurrió y proponga una hipótesis de contención.
5. Contrastar la reconstrucción de la IA contra lo que el equipo efectivamente ejecutó en el punto 1: señalar por escrito qué acertó la IA, qué omitió, y si propuso alguna afirmación que el log no sustenta.
6. Documentar la decisión final de contención —qué haría el equipo en un caso real: bloquear el origen, revertir un rol, forzar el cambio de contraseñas, u otra medida— dejando explícito que esa decisión es del equipo, tomada a partir de evidencia, y no una acción ejecutada automáticamente a partir de lo que sugirió la IA.

Lectura crítica: qué se evalúa en esta parte
No se evalúa únicamente si el simulacro corre y si la IA da una respuesta razonable. Se evalúa, sobre todo, la capacidad del equipo para detectar en qué se equivocó la reconstrucción de la IA —una fecha mal inferida, una causa asumida sin evidencia, un paso que el log no registra— y para tomar la decisión de contención con criterio propio.

Entregable de esta parte: *El script del simulacro, el fragmento de log anonimizado, la respuesta completa de la IA, el contraste escrito entre esa respuesta y lo realmente ejecutado, y el informe de contención con la decisión final del equipo.*

PARTE D. Protección de datos personales: anonimización y verificación de equivalencia

Esta parte retoma la técnica de anonimización de la Clase 2 y la lleva un paso más allá: no alcanza con anonimizar, hay que demostrar que anonimizar no distorsiona lo que la base tiene para decir en conjunto.

1. Construir (o reutilizar y actualizar, si ya existe de la Clase 2) la tabla usuario_anon, con datos sintéticos deterministas siguiendo el patrón visto en clase.

2. Especificarle a OpenCode, con precisión, una consulta de agregación sobre usuario_anon —por ejemplo, cantidad de usuarios por rol y por mes de alta— y pedirle la consulta equivalente sobre la tabla real usuario.
3. Ejecutar ambas consultas y verificar que sus resultados agregados coinciden: mismos conteos, mismos agrupamientos. Adjuntar ambos resultados uno junto al otro.
4. Redactar, en dos o tres oraciones, qué información concreta de Food Store no debería salir nunca hacia un asistente de IA sin pasar antes por este proceso de anonimización.

Entregable de esta parte: *El script de creación o actualización de usuario_anon, las dos consultas generadas, y la evidencia de que sus resultados agregados son equivalentes.*

PARTE E. Cierre: consolidación de la bitácora de IA

El trabajo cierra con una revisión propia del proceso, no solo del resultado técnico.

1. Consolidar en BITACORA.md todos los registros de las Partes A a D: cada prompt relevante enviado a OpenCode, un resumen de la respuesta recibida, y la validación o corrección humana aplicada.
2. Señalar, entre todos los registros, cuál considera el equipo el caso más significativo de un error o una imprecisión de la IA detectado durante el trabajo, y explicar en un párrafo breve cómo se lo detectó.
3. Revisar el historial de Git del repositorio y confirmar que refleja el avance por partes, con mensajes de commit legibles. Si el historial quedó desordenado, dejar una nota breve en BITACORA.md explicando qué se haría distinto la próxima vez.

Entregable de esta parte: *BITACORA.md consolidada, con el análisis del punto 2 incluido, y el repositorio Git con el historial completo del trabajo.*

Entregables consolidados

El repositorio Git entregado debe contener, como mínimo, la siguiente estructura:

```
food-store-tp-u5/
├── sql/
│   ├── roles.sql
│   ├── auditoria_test.sql
│   ├── simulacro_incidente.sql
│   └── usuario_anon.sql
├── docs/
│   ├── kiro_roles_spec/          (requirements y design)
│   ├── informe_auditoria.md
│   ├── log_auditoria_prueba.txt
│   ├── log_simulacro_anonimizado.txt
│   ├── informe_incidente.md
│   └── verificacion_equivalencia.md
└── BITACORA.md
```

Criterios generales de evaluación

La corrección de este trabajo, y la devolución que cada equipo recibirá, considera las siguientes dimensiones. No sustituyen la rúbrica detallada que la cátedra aplica sobre cada entrega, pero orientan dónde poner el esfuerzo:

Dimensión	Qué se observa
Diseño de permisos (Parte A)	Que los roles y privilegios estén justificados y verificados, no copiados sin revisión.
Auditoría (Parte B)	Que la configuración esté correctamente activada y el log capturado sea legible y pertinente.
Simulacro e incidente (Parte C)	Que el escenario reproduzca la secuencia pedida, que el escalamiento falle si el diseño de permisos es correcto, y que exista lectura crítica real sobre la respuesta de la IA.
Protección de datos (Parte D)	Que la anonimización sea efectiva y que la equivalencia de resultados esté demostrada, no solo afirmada.
Uso crítico de la IA	Que la bitácora muestre casos reales de validación o corrección humana, no un registro vacío de trámite.
Trabajo en Git	Historial de commits legible, con avance por partes.

Formato y plazo de entrega

- La entrega se realiza mediante el enlace al repositorio Git (o el archivo comprimido del repositorio, si el equipo no dispone de un servicio de hosting) según el canal y el plazo publicados en el aula virtual.
- Los informes escritos (Parte B punto 4, Parte C puntos 5 y 6, Parte D punto 4) pueden redactarse en Markdown o en el formato que el equipo prefiera, siempre dentro del repositorio.
- Ante cualquier duda sobre el alcance de una consigna, se prioriza siempre trabajar exclusivamente sobre el propio entorno de desarrollo, nunca sobre sistemas de terceros.

Este trabajo práctico es la instancia integradora de la Unidad 5. Los artefactos que produce —el esquema de roles verificado, la auditoría activa, y el criterio para reconstruir un incidente con ayuda de IA sin delegarle la decisión final—